

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	M.MANASA	Reg. No.:	192425189
Section / Batch:	2024 AIML	Date:	26/8/2026

Code of Conduct

I _____M.MANASA(192425189)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____m.manasa_____

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	

Pseudocode Structure	10 %	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

1. Problem Understanding

A traffic monitoring system must process video frames to detect vehicles entering a defined Region of Interest (ROI), track their movement across frames, and count them exactly once as they cross a counting line, while displaying the running count and drawing visual overlays.

1.1 Inputs

- Video stream / file: sequence of frames $F(t)$, $t = 0, 1, 2, \dots$
- ROI definition: polygon or rectangle coordinates on the frame
- Counting line: a virtual line (or line segment) inside the ROI
- Detection model / background subtractor parameters (confidence threshold, min contour area) • Tracker parameters (max distance for association, max frames to keep a lost track)

1.2 Outputs

- Annotated video frames showing: ROI boundary, counting line, bounding boxes, track IDs, trajectories • Real-time vehicle count displayed on screen
- Final count summary (optional log/CSV of vehicle crossing timestamps)

1.3 Constraints

- Avoid double counting → each tracked vehicle must be counted only once (using a unique track ID and a "counted" flag)
- Handle multiple vehicles simultaneously → the system must support many concurrent tracks with independent identities
- Must work frame-by-frame in near real time
- Must handle vehicles entering/leaving the ROI at different times (occlusion, temporary disappearance)

2. Algorithm Design – System Overview

The system is organized into four cooperating modules, executed once per frame inside a main loop:

- Frame Acquisition – read frames from the video source
- Vehicle Detection – detect vehicle candidates within the ROI
- Multi-Object Tracking – associate detections with existing tracks across frames (data association) • Counting & Visualization – increment the count when a track crosses the counting line, and draw all overlays

3. Complete Pseudocode

3.1 Data Structures

```
STRUCTURE Track:
  id // unique integer ID
  centroid // (x, y) current center position
  bbox // (x1, y1, x2, y2) bounding box
  trajectory // list of past centroids
  age // frames since last matched detection
  hits // number of consecutive successful matches
  counted // BOOLEAN, TRUE once vehicle has been counted
GLOBAL VARIABLES:
  trackList = empty list of Track // all currently active tracks  nextTrackID =
0
  vehicleCount = 0
  MAX_AGE = 10 // frames a track may go unmatched before deletion  MATCH_DIST = 50 // max
centroid distance (pixels) to associate a detection
```

3.2 Main Program

```
BEGIN VehicleCountingSystem
  video <- OPEN_VIDEO_SOURCE(path)
  ROI <- DEFINE_REGION_OF_INTEREST()
  countLine <- DEFINE_COUNTING_LINE(ROI)

  WHILE video HAS next frame DO
    frame <- READ_NEXT_FRAME(video)
    IF frame IS NULL THEN
      BREAK // end of video
    END IF

    detections <- DETECT_VEHICLES(frame, ROI)
    UPDATE_TRACKS(detections)
    COUNT_CROSSINGS(countLine)
    DRAW_VISUALS(frame, ROI, countLine)

    DISPLAY(frame)
    IF USER_PRESSED('q') THEN
      BREAK
    END IF
  END WHILE

  RELEASE(video)
  CLOSE_ALL_WINDOWS()
END VehicleCountingSystem
```

3.3 Module A — Vehicle Detection

Detects candidate vehicles only inside the ROI to reduce false positives and processing cost.

```
FUNCTION DETECT_VEHICLES(frame, ROI):
  maskedFrame <- APPLY_ROI_MASK(frame, ROI)
  gray <- CONVERT_TO_GRAYSCALE(maskedFrame)
```

```

fgMask <- BACKGROUND_SUBTRACTOR.APPLY(gray) // or a trained model e.g. YOLO/Haar fgMask
<- MORPHOLOGICAL_OPEN(fgMask) // remove noise fgMask <- MORPHOLOGICAL_DILATE(fgMask) //
fill gaps

contours <- FIND_CONTOURS(fgMask)
detections <- empty list

FOR EACH contour IN contours DO
  area <- CONTOUR_AREA(contour)
  IF area >= MIN_VEHICLE_AREA THEN
    (x, y, w, h) <- BOUNDING_RECT(contour)
    centroid <- (x + w/2, y + h/2)
    IF POINT_INSIDE_POLYGON(centroid, ROI) THEN
      ADD (bbox=(x,y,x+w,y+h), centroid=centroid) TO detections
    END IF
  END IF
END FOR

RETURN detections
END FUNCTION

```

3.4 Module B — Multi-Object Tracking (across frames)

Associates new detections with existing tracks using nearest-centroid matching, creates new tracks for unmatched detections, and removes stale tracks — enabling correct handling of multiple simultaneous vehicles.

```

FUNCTION UPDATE_TRACKS(detections):
  matchedTrackIDs <- empty set
  matchedDetections <- empty set

  // Step 1: Associate each existing track with the closest unmatched detection
  FOR EACH track IN trackList DO
    bestDetection <- NULL
    bestDistance <- INFINITY

    FOR EACH detection IN detections DO
      IF detection NOT IN matchedDetections THEN
        dist <- EUCLIDEAN_DISTANCE(track.centroid, detection.centroid) IF dist <
        bestDistance AND dist < MATCH_DIST THEN
          bestDistance <- dist
          bestDetection <- detection
        END IF
      END IF
    END FOR

    IF bestDetection IS NOT NULL THEN
      track.trajectory.APPEND(track.centroid)
      track.centroid <- bestDetection.centroid
      track.bbox <- bestDetection.bbox
      track.age <- 0
      track.hits <- track.hits + 1
      ADD track.id TO matchedTrackIDs
      ADD bestDetection TO matchedDetections
    ELSE

```

```

track.age <- track.age + 1 // no match this frame
END IF
END FOR

// Step 2: Create new tracks for detections that matched no existing track
FOR EACH detection IN detections DO
  IF detection NOT IN matchedDetections THEN
    newTrack <- CREATE Track(
      id = nextTrackID,
      centroid = detection.centroid,
      bbox = detection.bbox,
      trajectory = empty list,
      age = 0, hits = 1, counted = FALSE)
    ADD newTrack TO trackList
    nextTrackID <- nextTrackID + 1
  END IF
END FOR
// Step 3: Remove tracks that have been unmatched for too long (left ROI / occluded too long)
FOR EACH track IN trackList DO
  IF track.age > MAX_AGE THEN
    REMOVE track FROM trackList
  END IF
END FOR
END FUNCTION

```

3.5 Module C — Counting Logic (avoids double counting)

A vehicle is counted the moment its trajectory crosses the counting line, and only once — enforced by the per track "counted" flag.

```

FUNCTION COUNT_CROSSINGS(countLine):
  FOR EACH track IN trackList DO
    IF track.counted = FALSE AND LENGTH(track.trajectory) >= 1 THEN
      prevPoint <- LAST(track.trajectory)
      currPoint <- track.centroid

      IF LINE_SEGMENTS_INTERSECT(prevPoint, currPoint, countLine.p1, countLine.p2) THEN
        vehicleCount <- vehicleCount + 1
        track.counted <- TRUE // guarantees single count per vehicle
      END IF
    END FOR
  END FUNCTION

```

3.6 Module D — Visualization (drawing functions)

```

FUNCTION DRAW_VISUALS(frame, ROI, countLine):
  DRAW_POLYGON(frame, ROI, color = YELLOW, thickness = 2)
  DRAW_LINE(frame, countLine.p1, countLine.p2, color = RED, thickness = 2)

  FOR EACH track IN trackList DO
    DRAW_RECTANGLE(frame, track.bbox, color = GREEN, thickness = 2)
    DRAW_TEXT(frame, "ID:" + track.id, position = TOP_LEFT(track.bbox), color = WHITE)
    DRAW_CIRCLE(frame, track.centroid, radius = 4, color = BLUE, filled = TRUE)
  END FOR
END FUNCTION

```

```

FOR i FROM 1 TO LENGTH(track.trajectory) - 1 DO
DRAW_LINE(frame, track.trajectory[i-1], track.trajectory[i], color = CYAN, thickness
= 1)
END FOR
END FOR

DRAW_TEXT(frame, "Vehicle Count: " + vehicleCount,
position = (20, 40), color = WHITE, fontScale = 1, thickness = 2) END FUNCTION

```

4. Handling Constraints & Edge Cases (Completeness)

- Multiple simultaneous vehicles: trackList stores an independent Track object per vehicle; the nearest centroid association loop (3.4) matches many detections to many tracks each frame.
- Avoiding double counting: each Track carries a persistent counted flag that is set once and checked before every crossing test (3.5), so re-entering the same frame region cannot increment the count again.
- Temporary occlusion: a track is kept alive (not deleted) for up to MAX_AGE unmatched frames, so a briefly hidden vehicle is not lost and re-counted as a "new" vehicle.
- Noise/false detections: minimum contour area filtering and ROI masking in Module A reduce spurious detections.
- Vehicles leaving without crossing the line: their tracks simply age out and are removed (Step 3, Module B) without affecting the count.
- Scalability: all loops operate on the current active track/detection lists only, keeping per-frame cost proportional to the number of vehicles visible, not the full video history.

5. Time Complexity (per frame)

Let n = number of existing tracks, m = number of new detections. Detection: $O(\text{pixels})$ for background subtraction + $O(\text{contours})$. Association (Module B): $O(n \times m)$ using nearest-centroid matching. Counting and drawing: $O(n)$. Overall per-frame complexity is $O(n \times m)$, which is efficient since n and m (vehicles per frame) are small relative to frame resolution.